

Random Maze Solver

Parallelizzazione con OpenMP della risoluzione concorrente di
un labirinto

Lorenzo Cappellini

Parallel Computing - A.A. 2025/2026

Indice

1	Introduzione	3
1.1	Obiettivo	3
1.2	Approcci utilizzati	3
2	Architettura e implementazione	5
2.1	Flusso del programma	5
2.2	Classi	5
2.3	Generazione del labirinto	6
2.4	Parallelizzazione con OpenMP	6
2.4.1	Sincronizzazione	6
3	Analisi delle performance	7
3.1	Difficoltà riscontrate	7
3.2	Setup hardware	7
3.3	Risultati	8
3.3.1	Numero di iterazioni	8
3.3.2	Speedup rispetto alla singola particella	9
3.3.3	Speedup rispetto al singolo thread	10
3.3.4	Speedup complessivo	11
4	Conclusioni	11

1 Introduzione

1.1 Obiettivo

Il progetto è stato sviluppato per l'assignment del corso di Parallel Computing, il quale richiede l'implementazione di un programma che sfrutti almeno uno degli approcci di parallelizzazione o vettorizzazione presentati durante il corso (OpenMP, vettorizzazione SIMD, CUDA), a partire da una versione sequenziale dello stesso algoritmo, corredata da un'analisi delle performance in termini di *speedup*.

Il problema scelto è il **Random Maze Solver**: dato un labirinto, l'obiettivo è individuare l'uscita simulando il movimento casuale di una "particella". Questa parte da una posizione iniziale all'interno del labirinto e si muove casualmente tra le celle; quando incontra un muro, la particella rimbalza. Per aumentare la probabilità di individuare rapidamente l'uscita, vengono "lanciate" simultaneamente un numero elevato di particelle, ciascuna delle quali segue il proprio percorso. La simulazione termina quando la prima particella riesce a uscire dal labirinto. A questo punto, il percorso seguito dalla particella vincente viene ricostruito tramite *backtracking*, permettendo di individuare il cammino che conduce all'uscita.

Il codice sorgente è disponibile su Github al seguente indirizzo:
<https://github.com/lcappellini/RandomMazeSolver>.

1.2 Approcci utilizzati

Dato che **ogni particella è indipendente** dall'altra, il problema si presta particolarmente alla parallelizzazione mediante **OpenMP**: ogni thread può occuparsi dell'avanzamento di una particella (o più) in modo indipendente.

L'unica forma di **sincronizzazione** necessaria riguarda l'individuazione della prima particella che raggiunge l'uscita e la conseguente terminazione di tutti i restanti thread.

Il carico di lavoro non è descritto da un'unica variabile ma da **due grandezze distinte**:

- il **numero di particelle** lanciate, che determina quanti tentativi indipendenti di ricerca casuale vengono effettuati.
Più particelle $\Rightarrow$ più probabilità di trovare un percorso migliore $\Rightarrow$ minor tempo di esecuzione;
- il **numero di thread** su cui tali particelle vengono effettivamente distribuite ed eseguite in parallelo.
Più thread $\Rightarrow$ più parallelismo $\Rightarrow$ minor tempo per far avanzare le particelle.

Per semplicità di gestione e implementazione non è specificato il numero totale di particelle, ma il numero di particelle che ogni thread lancia.



Figura 1: Immagine del labirinto con una particella che si muove randomicamente dalla partenza (in alto a sinistra) fino a raggiungere l'uscita (in basso a destra) in 350000 steps (caso ottimo)

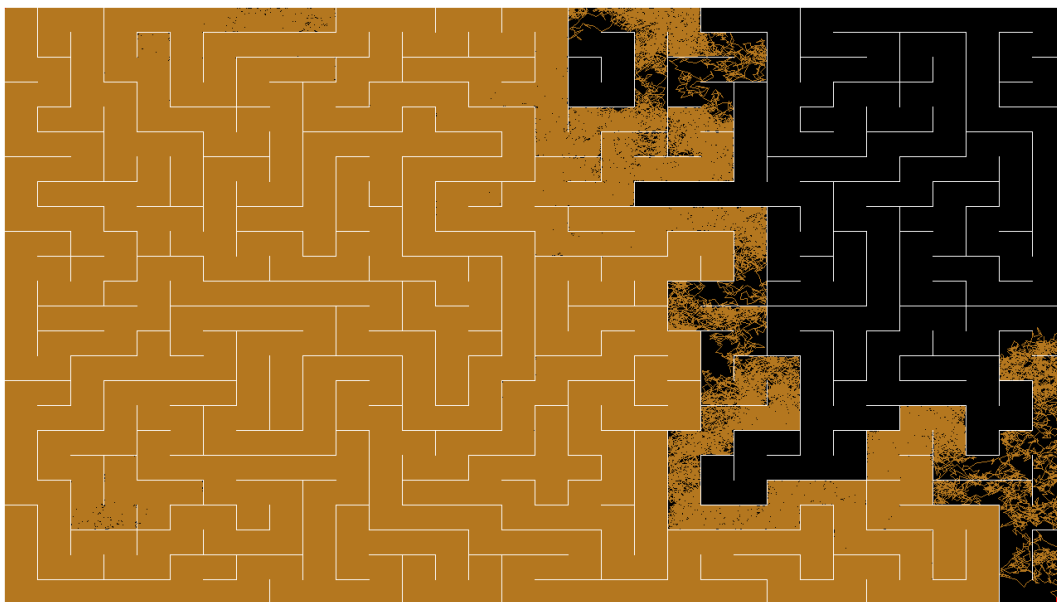


Figura 2: Immagine del labirinto con una particella che si muove randomicamente dalla partenza (in alto a sinistra) fino a raggiungere l'uscita (in basso a destra) in 5 milioni di steps (caso pessimo)

2 Architettura e implementazione

2.1 Flusso del programma

Per ciascuna coppia (numero di particelle per thread p , numero di thread n), il programma:

1. genera lo stesso labirinto di partenza, riutilizzato per tutte le misurazioni;
2. avvia n thread (tramite le direttive OpenMP e `num_threads(n)`), ognuno dei quali lancia p particelle indipendenti $\Rightarrow$ totale $p * n$ particelle;
3. avanza le particelle con step random ottenuti dagli `StepGenerator`, in un range $(-step, +step)$ per le due coordinate x e y ;
4. appena una particella arriva all'uscita (entra nella regione di arrivo), misura il tempo trascorso tramite `std::chrono::high_resolution_clock`, e ritorna anche il numero di iterazioni impiegate;
5. ripete la misurazione più volte per ciascuna coppia (p, n) ;
6. salva le misurazioni effettuate in un file `.csv`.

La griglia di configurazioni testate copre $p \in 1, \dots, 20$ e $n \in 1, \dots, 16$, per un totale di 320 configurazioni distinte, ciascuna ripetuta 100 volte.

2.2 Classi

Per modellare il problema sono state create varie classi:

- **Point**: struttura dati elementare che rappresenta un punto geometrico, con coordinate (x, y) .
- **Rect**: rappresenta un rettangolo geometrico, utilizzato per delimitare la regione di arrivo (uscita del labirinto); tramite il metodo `inside(Point&)` è possibile verificare se un **Point** è al suo interno.
- **Path**: identifica il percorso della particella tramite un vector di **Point**.
- **StepGenerator**: implementa un generatore di numeri casuali (RNG) secondo una distribuzione uniforme in un range specificato.
- **Maze**: rappresenta il labirinto, suddiviso per semplicità in celle rettangolari, ciascuna delle quali specifica l'eventuale presenza di un muro nei quattro lati. Internamente si occupa di:
 - generare il labirinto randomicamente tramite l'algoritmo di *Recursive Backtracking*;
 - validare i movimenti delle particelle calcolando gli eventuali rimbalzi sui muri;
 - disegnare il labirinto e i percorsi trovati su un'immagine (tramite la libreria **OpenCV**);
- **MazeSolver**: implementa la logica di risoluzione vera e propria, facendo avanzare le particelle con movimenti rettilinei di lunghezza casuale finché uno di essi non raggiunge la regione di arrivo. Internamente suddivide il lavoro sul numero specificato di thread.

2.3 Generazione del labirinto

Il labirinto viene generato, in modo sequenziale, tramite l'algoritmo di **Recursive Backtracking**: partendo dalla cella (0,0), l'algoritmo visita ricorsivamente le celle adiacenti non ancora visitate in ordine casuale, rimuovendo il muro condiviso tra la cella attuale e quella appena raggiunta. Si ottiene quindi un labirinto in cui esiste un unico percorso dall'ingresso all'uscita.

In realtà il labirinto viene generato una volta sola e poi salvato su file. All'avvio successivo del programma il file viene ricaricato, in modo da avere sempre lo stesso identico labirinto e quindi eliminare questa variabilità ed avere risultati più consistenti.

2.4 Parallelizzazione con OpenMP

La parallelizzazione è ottenuta annotando il ciclo esterno sugli "agenti" con `#pragma omp parallel for`. In questo modo ogni agente viene assegnato a un thread differente ed esplora il labirinto in modo indipendente e concorrente rispetto agli altri, senza necessità di sincronizzazione sui dati del labirinto (in sola lettura per tutti i thread) né sui percorsi (ciascun agente scrive esclusivamente sul proprio `Path`, quindi non si verificano race condition in scrittura).

2.4.1 Sincronizzazione

L'unica variabile condivisa in scrittura tra i thread è il flag booleano `running`, utilizzato come condizione di terminazione: quando un agente raggiunge il traguardo lo imposta a `false` per segnalare la fine della ricerca a tutti gli altri thread (che lo controllano ad ogni iterazione).

Per sincronizzare questa variabile è possibile utilizzare direttive di OpenMP apposite (come `#pragma omp critical` o `#pragma omp atomic read/write`). Tuttavia, per una maggiore semplicità di utilizzo si è optato per la libreria C++ `atomic`, che permette di creare variabili atomiche thread-safe senza direttive pragma.

```
1 #pragma omp parallel for num_threads(n)
2 for (int t = 0; t < n; t++) { // 1 ciclo per ogni thread
3     // inizializzazione vettori di generatori, paths e iterazioni
4     while (running.load(std::memory_order_relaxed)) {
5         for (int i = 0; i < (int)p; i++) { // per ogni particella del thread
6             // genera prossimo punto (p2) e calcola i rimbalzi
7             // incrementa iterazioni
8             if (end.inside(p2)) { // entrata nella regione di arrivo
9                 bool expected = true;
10                if (running.compare_exchange_strong(expected, false, std::memory_order_relaxed)) {
11                    // salva iterazioni e percorso trovato
12                }
13                break;
14            }
15        }
16    }
17 }
18 // calcola tempo trascorso
19 // ritorna iterazioni e tempo
```

Listing 1: Snippet di codice (semplificato) che evidenzia il codice di sincronizzazione

3 Analisi delle performance

3.1 Difficoltà riscontrate

La natura del problema rende l'analisi dei risultati tutt'altro che semplice, soprattutto a causa della componente randomica dei movimenti delle particelle. Infatti molto spesso si ottengono risultati (tempi di esecuzione e numero di iterazioni) incoerenti tra loro in quanto l'arrivo della particella a destinazione dipende dalla "fortuna"/"sfortuna" nel trovare un percorso ottimo/pessimo, indipendentemente dal numero di particelle o di thread impiegati.

Per ovviare a questo problema è stato fatto un tentativo di rimuovere la componente random, fissando quindi un seed per l' RNG, ma a quel punto si perde completamente il fondamento del problema. Inoltre, aumentare il numero di particelle o thread, porta solo a un carico di lavoro superfluo dato che tutte le particelle si muovono secondo lo stesso identico percorso.

L'unica soluzione trovata per ridurre l'impatto della componente random dell'algoritmo è quindi quella di effettuare un grande numero di misurazioni, in modo da ridurre il peso statistico degli outlier. Per ricavare il valore finale è stata utilizzata la mediana, che da questo punto di vista è più robusta rispetto alla media.

3.2 Setup hardware

Componente	Modello
CPU	AMD Ryzen 7 3700X 8-Core Processor
RAM	2 x Corsair 8GB DDR4
GPU	NVIDIA RTX 3060 Ti
Sistema operativo	Windows 10

Tabella 1: Configurazione hardware/software utilizzata per i benchmark

Il Ryzen 7 3700X mette a disposizione 16 thread logici (8 core fisici, SMT abilitato): questo valore rappresenta il naturale punto di saturazione del parallelismo reale sfruttabile.

3.3 Risultati

3.3.1 Numero di iterazioni

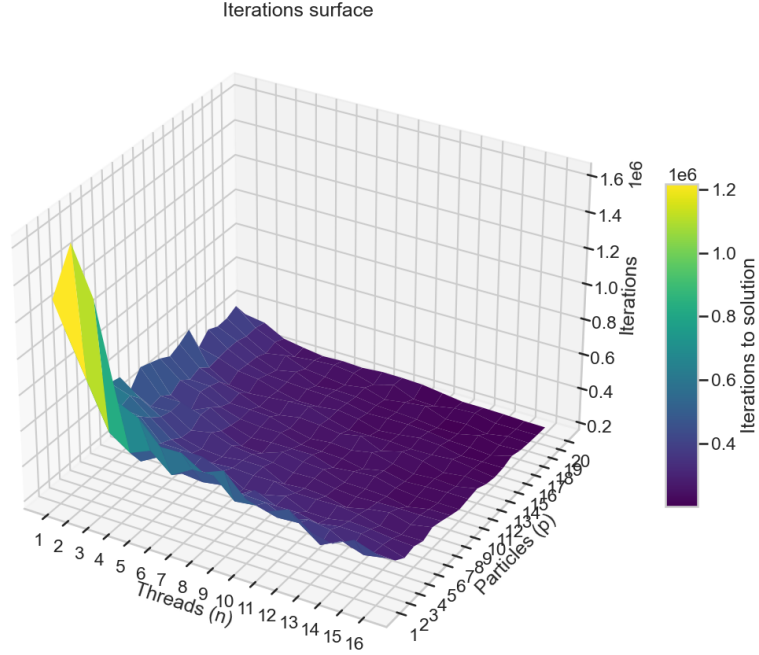


Figura 3: Grafico mesh del numero di iterazioni/step del percorso trovato al variare del numero di thread e del numero di particelle per thread

Dal grafico si nota come il numero di step del percorso trovato dopo la ricerca diminuisca notevolmente all'aumentare del numero di particelle. Anche l'aumento dei thread aumenta il numero di particelle totale: $p_{tot} = p * n$.

Questo conferma l'ipotesi teorica in cui avere più particelle aumenta la probabilità di trovare un percorso migliore, infatti le iterazioni sembrano seguire una decrescita proporzionale a $1/P_{tot}$.

3.3.2 Speedup rispetto alla singola particella

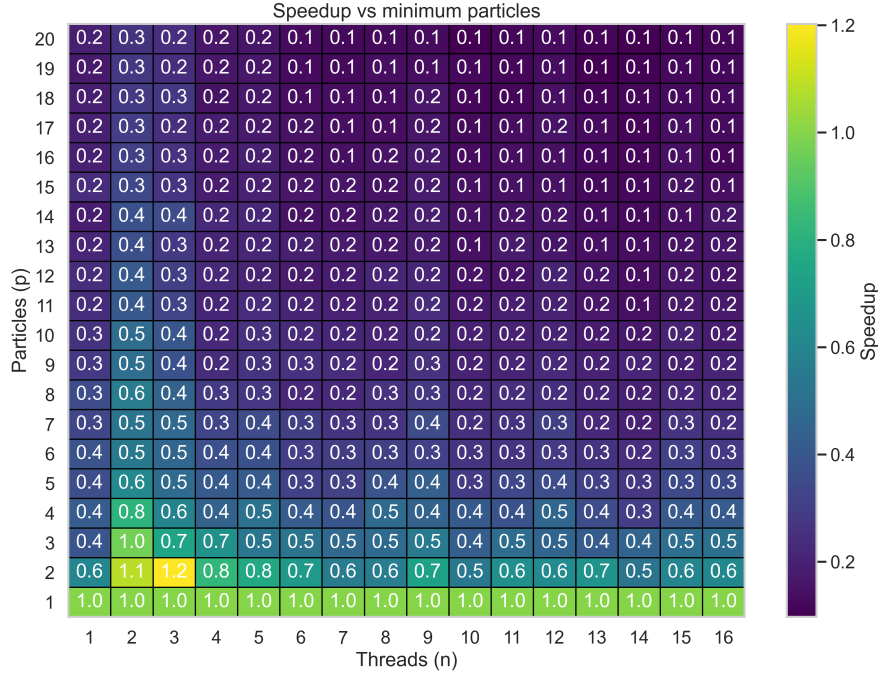


Figura 4: Heatmap dello speedup, calcolato rispetto a $T(1, n)$

Formula: $S(p, n) = T(1, n)/T(p, n)$

Questa metrica permette di valutare l'effetto della variazione del numero di particelle rispetto al numero di thread n fissato. Aumentando il numero di particelle, il carico di lavoro del thread aumenta linearmente, quindi se generalmente per arrivare alla soluzione un thread impiega (in media) x step di una particella, con n particelle dovrà eseguire $n * x$ iterazioni totali.

È vero che un numero di particelle maggiore aumenta la probabilità di trovare un cammino più veloce, ma non compensa a sufficienza il costo del carico aggiuntivo, però giustifica la decrescita non lineare dello speedup (fissato n).

Valori come $S(2, 3) = 1.2$ sono da considerarsi anomalie di misurazione dovute alla grande varianza dei dati a causa della componente random alla base (vedi 3.1).

3.3.3 Speedup rispetto al singolo thread

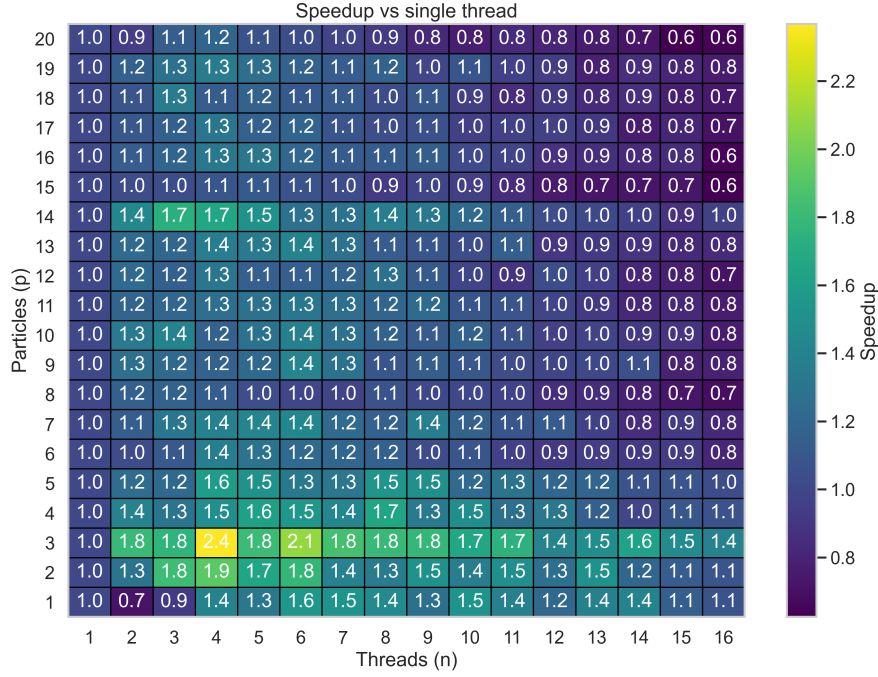


Figura 5: Heatmap dello speedup, calcolato rispetto a $T(p, 1)$

Formula: $S(p, n) = T(p, 1)/T(p, n)$

Valutando lo speedup relativo all'introduzione di ulteriori thread nella ricerca, è evidente che si ottiene un beneficio: avere più agenti che agiscono quindi su nuove particelle, aumenta la probabilità di trovare un percorso alla destinazione più breve. Questo miglioramento è praticamente gratuito finché ci sono core fisici disponibili nella CPU.

L'unico costo aggiuntivo è quello di creazione, gestione e sincronizzazione dei thread (assente del tutto solo nel caso in cui $n = 1$): per questo motivo, come si vede dalla heatmap, avere 1 particella per thread con soltanto 2 thread, non rende giustizia a questo costo aggiuntivo e si ottiene di fatto un peggioramento delle prestazioni. In generale i valori massimi di speedup si raggiungono con un numero di thread da 6 a 8, dopodiché i valori tornano a scendere. Questo comportamento è coerente con l'architettura della CPU utilizzata, che offre 8 core fisici e 16 thread logici (SMT): fino a $n = 8$ ogni thread ha a disposizione un core fisico dedicato, e il guadagno di parallelizzazione è reale. Superato questo limite, i thread aggiuntivi condividono le cache L1/L2 dello stesso core fisico tramite SMT, che genera quindi situazioni di contesa.

3.3.4 Speedup complessivo

$$S(p, n) = T(1, 1)/T(p, n).$$

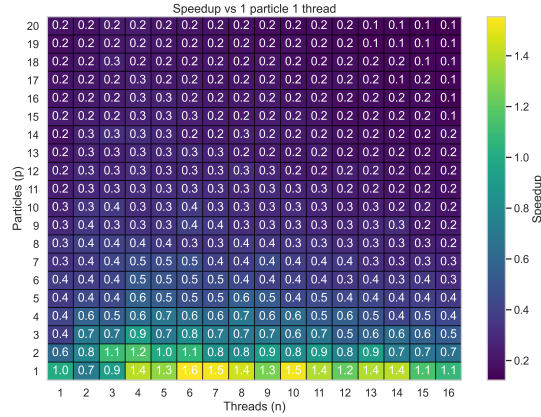


Figura 6: Heatmap dello speedup, calcolato rispetto a $T(1, 1)$

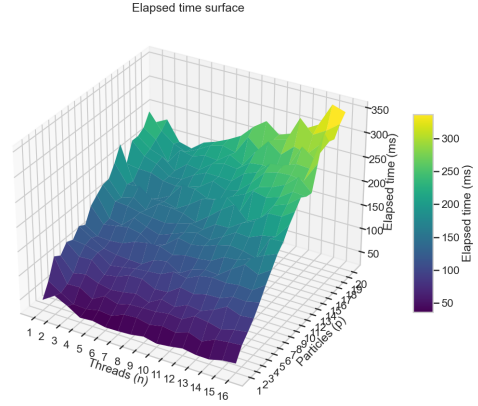


Figura 7: Grafico mesh dei tempi misurati

Formula: $S(p, n) = T(1, 1)/T(p, n)$

L'heatmap qui sopra mostra lo speedup netto rispetto al caso base (1 particella, 1 thread). Da questo è possibile notare il peso reale dell'aumento del numero delle particelle piuttosto che quello di thread: a livello di prestazioni è molto più conveniente aggiungere nuovi thread, che lavorano comunque su un numero limitato di particelle, piuttosto che avere molte particelle su pochi thread.

Nel grafico dei tempi si nota come il tempo assoluto richiesto aumenti sia con p che con n . Nuovamente i valori di picco si trovano per valori di thread intorno a 8, per i motivi spiegati nella sezione 3.3.3.

4 Conclusioni

Visti i risultati ottenuti, è possibile concludere che:

- l'elevata variabilità intrinseca del problema rende molto difficile studiarne le prestazioni.
- aumentare il numero di particelle riduce fortemente il numero di step del percorso finale trovato;
- aumentare il numero di thread produce uno speedup reale, che però si degrada superato il numero di thread fisici della macchina;

Pertanto la configurazione ottimale per questo algoritmo, con un buon compromesso tra lunghezza del percorso e tempi di esecuzione, si trova con un numero basso di particelle per thread ($p = 4$) e un numero di thread entro pari ai core fisici disponibili ($n = 8$ sulla CPU utilizzata).